

String art from photos, explained

This note explains what the project does, where it came from, what was wrong with the method everyone uses, and what we changed. All numbers come from `docs/report/report.md`.

The idea in one paragraph

You hammer a few hundred pins around a circular frame. You tie a black thread to one pin and wind it from pin to pin in straight lines, a few thousand times. Where many lines cross, the board looks dark. Where few cross, it stays white. Pick the right sequence of pins and a portrait appears. The hard part is the picking. Every line darkens everything it passes through, a line can't be taken back once wound, and the thread has to stay one continuous piece. Our program takes a photo and computes that sequence.

Terminology

term	meaning
pin	A nail on the frame rim. We number them clockwise from the top. Our frame has 300 on a 700 mm circle.
chord or line	One straight stretch of thread between two pins.
pin sequence	The ordered list of pins the thread visits. This is the real output, because it's what you wind.
canvas	The simulated image, 600 by 600 pixels by default.
target	The preprocessed photo the solver tries to match, stored as darkness from 0 for white to 1 for black.
opacity	How much of one pixel a single thread covers. It equals thread width divided by pixel size. A 0.25 mm thread on a 700 mm frame at 600 px gives about 0.21.
multiplicative composition	How overlapping threads stack. Each new thread darkens what's left of the white, $d \leftarrow d + a(1 - d)$, so a pixel never goes past black.
importance map	A weight per pixel. High weight means "get this pixel right", like the eyes and mouth.
greedy solver	An algorithm that picks the single best next line at each step and never looks back.
refinement	A second pass that edits the pin sequence after the greedy pass to fix its mistakes.

term	meaning
error E	How far the canvas is from the target, $E = \sum W (t - d)^2$, where W is importance, t is target darkness and d is canvas darkness.
SSIM	Structural similarity, a standard image-quality score from 0 to 1 that tracks how alike two images look. Higher is better.
PSNR	Peak signal-to-noise ratio in decibels. Higher is better. It is cruder than SSIM.
σ_2, σ_4	We blur both images with a Gaussian of 2 or 4 pixels before scoring. People look at string art from a few metres away, where threads blend together. The blur simulates that.
face SSIM	SSIM computed only inside the face outline.
ΔE_{2000}	CIEDE2000, a colour-difference formula built to match human perception. Lower is better. Around 2 is barely noticeable.
YuNet	A tiny face detector that ships with OpenCV, 0.2 MB.
LBF landmarks	68 points on the face, covering jaw, brows, eyes, nose and mouth, from OpenCV's Facemark model.
CLAHE	Contrast-limited adaptive histogram equalization. It boosts contrast locally instead of across the whole image.
bilateral filter	A blur that smooths flat areas but keeps edges sharp.
spectral-residual saliency	A fast method that guesses which parts of an image draw the eye.
Floyd–Steinberg dithering	A way to fake in-between colours with a pattern of a few solid ones, like a newspaper photo.
gamut	The set of colours you can actually make. With coloured thread on a white board you can only reach mixes of the board and the threads.

Previous work

Petros Vrellis made computational string art famous in 2016 with "A New Way to Knit". His pieces used thousands of chords on a circular loom, and the method behind them is a greedy loop.

The open version most people copy is the "Computational Thread Art" post on LessWrong. It works like this. Start at a pin. Look at every line you could draw from it and pick the one through the darkest remaining pixels. Subtract a fixed amount of darkness along that line. Move to the far pin and repeat, usually about 3,000 times. It precomputes line coordinates

and uses numpy, so a run takes about 10 seconds. It supports importance weights that you paint by hand, positive for regions to emphasize and negative for regions to keep clear. For colour, it dithers the photo into one image per thread colour, solves each one on its own, and layers the results. This is the baseline we compare against.

Birsak, Rist, Wonka and Musialski published "String Art: Towards Computational Fabrication of String Images" in Computer Graphics Forum in 2018. They model viewing distance properly by comparing a fine canvas to a coarse target, and they use an optimizer that can both add and remove lines. It's more principled than greedy, and much slower. We took their viewing-distance idea in a cheap form.

"Automated String Art Creation", IEEE Xplore document 10844087, is about the physical side. It covers CNC machines, 3D printing and tensioning thread with an Arduino. We used it as background for building a real piece.

What's wrong with the previous method

We found four problems with the LessWrong greedy method. Each of our main changes fixes one.

1. The darkness it subtracts per line is a made-up constant. It doesn't come from the thread or the frame, so the simulation and a real piece can disagree, and someone has to tune it.
2. You have to guess how many lines to draw. Too few and the image is faint. Too many and it goes muddy, because the method keeps subtracting darkness that isn't there. Quality peaks and then collapses, and the peak is in a different place for every photo.
3. Faces come out murky. The method treats a pixel of background the same as a pixel of eye. You can fix that with importance masks, but someone has to paint them for each photo.
4. Colour is solved one thread at a time. Each colour layer ignores the others, and dithering throws away detail before the solver even starts.

Greedy also has a deeper flaw. Once it draws a bad line, the line stays. Nothing goes back and fixes it.

What we changed

1. A physical thread model and a solver that stops by itself

We model the thread as it really is. Each thread covers a fraction of a pixel, the opacity, set by thread width and frame size. Overlapping threads stack multiplicatively. The solver, the renderer and the visualizer all share this one model, so what the solver optimizes is exactly what you see.

At each step the solver scores every legal line from the current pin by the exact drop in weighted error it would cause, $\Delta E = \sum W[(t - d)^2 - (t - d')^2]$. It draws the best one. When no line lowers the error, it stops. So the photo decides the line count, not the user.

Some rules keep the result buildable. Lines can't join pins that are too close together. The thread can't go straight back the way it came. No chord gets used more than twice.

A numba kernel scores all the candidates in parallel. It draws each line on the fly, so there's no big cache of precomputed lines. A greedy run takes about 0.9 seconds. One of the tests checks that the gains the solver adds up equal the error drop of the final rendered image, to within 1e-6.

2. Face-aware preprocessing and automatic importance maps

Before solving, OpenCV prepares the photo.

1. YuNet finds the face. We crop a square 1.8 times the face height, nudged slightly down. With no face, we crop the centre.
2. We fix exposure only when the photo is actually bad. That means a narrow brightness range, no real black or no real white. Our first version corrected every photo, and it made good photos worse. After stretching the levels, a gamma correction pulls the median toward mid-grey, which rescues washed-out photos.
3. A bilateral filter removes noise without softening edges. CLAHE adds local contrast.

Then we build the importance map from three parts. The 68 landmarks give the face outline at weight 0.5 and the eyes, brows, nose and mouth at 1.0. Scharr edges add 0.5. Saliency adds 0.25. Every pixel keeps a floor of 0.1, so the background still counts a little.

The map switches on only when we find a face. On animals and objects it didn't help.

Because the error is symmetric, a high weight on a light feature like an eye white or teeth also keeps stray threads off it. That does the job of the hand-painted negative masks in the old method, with nobody painting anything.

3. Path refinement that can undo mistakes

After the greedy pass, refinement walks the pin sequence and tries three edits.

edit	before	after
delete	$a \rightarrow b \rightarrow c$	$a \rightarrow c$
reroute	$a \rightarrow b \rightarrow c$	$a \rightarrow b' \rightarrow c$
insert	$a \rightarrow b$	$a \rightarrow x \rightarrow b$

Each edit keeps the thread in one continuous piece, so you can still wind it.

The trick that makes this cheap comes from the multiplicative model. You can remove a line exactly by dividing its effect back out, $d \leftarrow 1 - (1 - d)/(1 - a)$. So we take the old lines out, score the edit, and apply it only if the error drops. Otherwise we put the old lines

back. The error never goes up. Two sweeps is the default, and the full pipeline runs in about 8 seconds.

4. A joint colour solver with a palette that can reach the photo's colours

Each colour is its own physical thread with its own current pin. A coloured thread blends over whatever is beneath it, $C \leftarrow C(1 - a) + c \cdot a$. With black thread on white this is exactly the grayscale model, and a test checks that.

At every step the solver picks the best colour and line together, scored by drop in RGB error. So the layers know about each other. Each colour stays in use for at least 100 lines. That keeps spool changes to about 33 per piece instead of about 144, and it costs nothing in quality.

Picking the threads took two tries. Our first idea, from the original abstract, was k-means clustering on the photo's colours, snapping each centre to the nearest real thread. That kept dull, average tones and missed saturated ones. A boy's yellow shirt came out tan. The fix starts from what's physically possible. Thread on a white board can only make colours inside the convex hull of the board and the threads. Starting from black, we greedily add the thread from a list of 16 real ones that brings the photo's colours closest to that hull. Now the shirt gets yellow thread.

We dropped Floyd–Steinberg dithering for the same reason. The joint solver doesn't need it.

5. Output you can build from, and a fair test

Every run writes `instructions.txt`. It has the pins numbered clockwise, the winding list in blocks of 100 lines, and the running thread length. Colour runs also list how much thread each spool needs and which pin to tie on at. `stringart kit` prints a full-size pin template on A4 tiles plus a shopping list. `stringart wind` steps through the winding one line at a time and remembers where you stopped.

The one physical number the simulation depends on is opacity. `stringart calibrate` measures it. You wind a test pattern of about 250 lines and photograph it. The program finds the frame with a Hough circle, straightens it, and fits the opacity that best matches the photo. On synthetic photos it gets within 0.021 of the true value.

We set rules to keep the comparisons honest.

- All methods score against the same face-centred crop.
- We score exposure faults against the clean original. Scoring against the damaged input would punish the method for fixing it.
- Every image uses the same defaults: 600 px, 256 pins, opacity 0.2. Nothing is tuned per image.
- A held-out set of 12 photos and 6 synthetic faults checks the decisions made on the main set. We picked it by a fixed rule without looking. In each Wikimedia category we skipped the first 60 images, which we had already browsed, and took the next openly licensed ones that passed the face filter.

Results

The test set has 30 images. 27 are openly licensed from Wikimedia Commons and 3 are synthetic exposure faults of one portrait. By category there are 17 faces, 4 hard cases, 7 animals and 2 objects.

Solver alone

method	SSIM σ^2	PSNR σ^2	time, serial
old greedy, 3,000 lines as published	0.616	17.29	8.9 s
old greedy, same line count as ours	0.624	18.14	8.5 s
our solver	0.643	18.66	4.7 s

At the same line count ours wins on 29 of 30 images. With parallel scoring it takes 0.9 seconds. The SSIM-versus-lines plot is the clearest picture of the difference. The old method peaks and then falls apart as it over-darkens. Ours levels off, and it stops on its own near the top.

To be fair to the old method, if you pick its line count per image by looking at the results, it can beat ours slightly on textured images with no face. The cat gained 0.011 and the lighthouse 0.017. On faces ours wins easily, by 0.027 and 0.120 in the same test.

Preprocessing and importance

Compared with the solver alone, the full pipeline raises face SSIM by 0.060 on all 22 face images. Whole-frame SSIM drops by 0.007. We chose that trade on purpose, because it moves threads from the background to the face. Against the old published method, the full pipeline gains 0.051 face SSIM on 21 of 22 face images and 0.021 overall.

Exposure

Scored against the clean photo, with SSIM σ^2 :

fault	no fix	level stretch	stretch and gamma, our default	clean input
underexposed	0.614	0.661	0.635	0.639
low contrast	0.551	0.623	0.635	0.639
washed out	0.511	0.548	0.645	0.639
held-out, mean of 6	0.591	0.640	0.650	0.657

The trigger left all 27 natural photos in the main set alone. On the held-out set it left all 12 natural photos alone and caught 5 of 6 faults. The one it missed was the mildest underexposure, which already scored above its own clean original. Gamma costs a little on

underexposed photos, but it wins on average, and we didn't retune it after seeing the held-out numbers.

Refinement

	SSIM σ^2	face SSIM σ^2	lines	time
greedy only	0.637	0.689	2,786	1.5 s
plus 2 sweeps, our default	0.647	0.704	3,237	10.2 s
plus 3 sweeps	0.649	0.707	3,262	14.9 s

These timings come from a Docker container with 8 threads. On the 16-thread laptop the default takes about 8 seconds. Refinement helps most on detailed faces where greedy stopped early. The athlete photo gained 0.067 and the elderly man 0.074.

Colour, on 12 colourful images

method	$\Delta E_{2000} \sigma^2$, lower is better	luminance SSIM σ^2	spool switches
old colour method, dither and solve each colour alone	16.36	0.503	3
ours, joint solver and reachable palette, the default	11.34	0.649	33
ours with k-means palette	12.56	0.663	39
ours, k-means palette, no spool limit	12.64	0.657	144
black thread only	16.00	0.602	0

Ours beats the old colour method on all 12 images, cutting ΔE by 5.0 and raising luminance SSIM by 0.15. The reachable palette beats k-means on colour error. It gives up a little light-and-dark structure to do it. The held-out set agreed, 11.14 against 12.59.

Building a real one

- Going past about 256 pins barely helps. SSIM σ^2 is 0.544, 0.593, 0.613 and 0.618 for 128, 200, 256 and 320 pins. Our 300-pin frame is already on the flat part.
- Thread width relative to frame size matters most. Halving the opacity, with a thinner thread or a bigger frame, lifts SSIM σ^2 from 0.613 to 0.688 and face SSIM from 0.698 to 0.784. It needs about twice the lines.
- A 3,000-line portrait on a 700 mm frame uses about 1.5 km of thread.

What it doesn't do yet

Everything so far is simulated. We haven't built a physical piece. The calibration, build kit and winding assistant are ready, but we've only tested them on synthetic data.

The solver minimizes squared error, which isn't the same as what looks good. On very detailed images it stops a little early, and refinement only partly makes up for it. In a few cases refinement lowers the error and SSIM drops slightly anyway.

Some categories are small. With 2 objects and 4 hard cases, we can't conclude much about them.

Next, we plan to build the 700 mm, 300-pin piece using `docs/BUILD_GUIDE.md`. On the software side we want a stopping rule based on perceived quality, a palette that balances colour against structure, and refinement for colour.

References

1. P. Vrellis, "A New Way to Knit", 2016.
2. "Computational Thread Art", LessWrong. <https://www.lesswrong.com/posts/a2v5Syk6gJs7HnRoq/computational-thread-art>
3. M. Birsak, F. Rist, P. Wonka, P. Musialski, "String Art: Towards Computational Fabrication of String Images", Computer Graphics Forum 37(2), 2018.
4. "Automated String Art Creation: Integrated Advanced Computational Techniques and Precision Art Designing", IEEE Xplore, document 10844087.
5. W. Wu, H. Peng, S. Yu, "YuNet: A Tiny Millisecond-level Face Detector", Machine Intelligence Research, 2023.
6. S. Ren, X. Cao, Y. Wei, J. Sun, "Face Alignment at 3000 FPS via Regressing Local Binary Features", CVPR 2014.
7. X. Hou, L. Zhang, "Saliency Detection: A Spectral Residual Approach", CVPR 2007.
8. K. Zuiderveld, "Contrast Limited Adaptive Histogram Equalization", Graphics Gems IV, 1994.
9. Z. Wang, A. Bovik, H. Sheikh, E. Simoncelli, "Image Quality Assessment: From Error Visibility to Structural Similarity", IEEE Transactions on Image Processing 13(4), 2004.
10. G. Sharma, W. Wu, E. Dalal, "The CIEDE2000 Color-Difference Formula", Color Research and Application 30(1), 2005.